

RPi Setup Guide

1. Setup and connect access point

**all parts in step 2 are mandatory unless specified as your choice*

1. Open the RPi network manager interface (from terminal)
 - a. `sudo nmtui`
2. Create a new network connection (the access point)
 - a. select <Edit a connection>; press enter
 - b. select <Add>
 - i. select <Wi-fi>
 - ii. enter a <Profile name> (your choice)
 1. eg: local_AP
 - iii. enter <Device>: wlan0
 - iv. enter a <SSID name> (your choice; Wi-Fi name)
 1. eg: phantasiai_dev
 - v. select a <Mode>: Access Point
 1. channel remains <automatic>
 - vi. select a <Security>: WPA2 Personal
 - vii. enter a <Password> (your choice)
 1. eg: Ph_dev2026!
 - viii. enter an <IPv4> option: change from Automatic to Shared
 - ix. select <Automatically connect>
3. Initialize access point from RPi
 - a. go to Wi-Fi networks (where you normally go to connect to a Wi-Fi)
 - b. select RPi network (SSID name)
4. [Optional]: To use Internet while RPi is an access point
 - a. plug cable from internet router into ethernet port
 - b. OTHERWISE: disconnect from the AP and connect to a Wi-Fi with Internet

ALTERNATIVELY: if you are experiencing problems with the access point and plan to run the prototype frontend application from the RPi itself, you can skip this step entirely

Ensure you are connected to the internet during package installation

2A. Download prototype and install dependencies:

1. download prototype: <https://github.com/RobertoFelipeSG/PhantasiAI/tree/main/API>
2. create virtual environment and install required dependencies
 - a. in terminal, ensure you are in home/[USERNAME]/PhantasiAI/API/
 - i. you should see: [USERNAME]@[HOSTNAME]: ~/PhantasiAI/API \$
 - b. `python3 -m venv venv`
 - c. `source venv/bin/activate`
 - d. `pip install -r requirements.txt`
 - e. follow 2B, 2C, 2D for details on how to install additional packages
3. create configuration file (contains all variables for prototype)
 - a. in terminal, ensure you are in home/[USERNAME]/PhantasiAI/API/
 - i. you should see: [USERNAME]@[HOSTNAME]: ~/PhantasiAI/API \$

- b. `cd config/`
- c. `cp config.yaml.example config.yaml`
- d. to edit configurations: `nano config.yaml`
 - i. ensure these variables align with your hardware:
 1. `default_serial_port / serial_port_A`: USB port for Ganglion
 - a. unplug dongle and run in terminal: `ls /dev/tty*`
 - b. plug in dongle and rerun: `ls /dev/tty*`
 - c. use name of device that appeared
 - i. usually `/dev/ttyUSB0` or `/dev/ttyACM0`
 2. `selected_channels`: Ganglion channel for electrodes
 - a. eg: channel 1 = [0]
 3. `mac_address`: Ganglion Mac Address to speed up connection
 - a. run in terminal: `sudo hcitool lescan`
 - b. a fast scan will occur, Ctrl+C to stop the scan
 - c. look for the line: [MAC_ADDRESS] Ganglion 1.3 (aeba)
 - i. or something similar
 - d. use mac address that appeared
 4. `gpio_chip`: primary GPIO controller
 - a. run in terminal: `gpiodetect`
 - b. use gpio chip with [rp1-gpio] label
 - i. eg: `gpiochip4 [rp1-gpio]` (54 lines)
 - ii. usually `/dev/gpiochip0`, but some RPi models use `/dev/gpiochip0`
 5. `gpio_pin`: GPIO pin in use by primary chip
 - a. run in terminal: `pinctrl`
 - b. use pin labelled with OUTPUT function
 - c. eg: GPIO line 17 corresponds to pin 12
 6. for other variables please see [Detailed Documentation](#)
 - ii. Ctrl+O → Enter → Ctrl+X to save changes and exit

2B. To install BrainFlow on RPi:

note: this is done using the venv within the prototype folder

1. install cmake (you can do this anywhere)
 - a. `sudo apt update`
 - b. `sudo apt install cmake`
 - c. [you make also need to do]:
 - i. `sudo apt install build-essential`
 - ii. `sudo apt install libusb-1.0-0-dev`
2. from home directory, create a temporary directory:
 - a. `cd /tmp`
3. clone the brainflow git repository into the temporary directory:
 - a. git clone <https://github.com/brainflow-dev/brainflow.git>
4. activate the virtual environment used to run the prototype:
 - a. example: `source /home/phantasiai_rpi500/PhantasiAI/venv/bin/activate`

5. go into the necessary directory and compile brainflow:
 - a. `cd brainflow/tools/`
 - b. `python3 build.py`
6. install python package for brainflow:
 - a. `cd ..`
 - b. `cd python_package/`
 - c. `python3 -m pip install -U .`
7. clean up
 - a. deactivate
 - b. `cd ..` (x2 to go back to /tmp directory)
 - c. `rm -rf brainflow`
8. final expected output: Successfully installed brainflow-0.0.1

2C. To handle “liblsl.so” not found error:

**this might appear after installing requirements and trying to run main.py*

1. Download (as zip) correct LSL package from: <https://github.com/sccn/liblsl/releases>
 - a. download: liblsl-1.16.2-bookworm_arm64.deb
2. Go into Downloads folder and make temporary folder:
 - a. `cd Downloads/`
 - b. `mkdir liblsl_extract`
3. Extract LSL package into folder:
 - a. `dpkg-deb -x liblsl-1.16.2-bookworm_arm64.deb liblsl_extract`
4. Find [liblsl.so](#) file:
 - a. `find liblsl_extract -name “liblsl.so”`
5. Copy into correct LSL folder:
 - a. example: `cp liblsl_extract/usr/lib/liblsl.so /home/phantasiai_rpi500/PhantasiaI/API/venv/lib/python3.13/site-packages/pylsl/lib/`
6. Cleanup:
 - a. `rm -rf liblsl_extract`

2D. [OPTIONAL] To install mosquito for client terminal outputs:

1. Install Mosquito broker via terminal:
 - a. `sudo apt update`
 - b. `sudo apt install mosquitto mosquitto-clients -y`
 - c. `sudo systemctl start mosquitto`
2. To make mosquito start automatically everytime you boot:
 - a. `sudo systemctl enable mosquitto`
3. Verify it is running
 - a. `systemctl status mosquitto`

3. Configure and run prototype setup script

1. Create setup script in home directory:
 - a. in terminal, ensure you are in home/[USERNAME]

- i. you should see: [USERNAME]@[HOSTNAME]: ~ \$
- b. `cp ~/PhantasiAI/API/config/setup_phantasi.sh.example ~/setup_phantasi.sh`
- c. to configure setup variables:
 - i. `nano setup_phantasi.sh`
 - ii. change [ACCESS_POINT] to your RPi access point name
 - iii. `Ctrl+O` → `Enter` → `Ctrl+X` to save changes and exit
2. Enable permissions and run script:
 - a. `chmod +x setup_phantasi.sh`
 - b. `./setup_phantasi.sh` (must be connect to the Internet to run this script)
3. Reconnect to access point + reboot system to update changes: `sudo reboot`

4. Using the prototype (as a developer)

1. Running the main application
 - a. all you need to do is power on your RPi; the application will automatically run
 - b. if an error occurs during main script execution, you will receive a notification
 - i. if you are receiving an error immediately from startup, ensure you are connected to the correct network (you check manually via the WiFi icon)
2. Using the application (via webbrowser)

note: device must be connected to the same network as the RPi

 - a. if using RPi access point: go to <http://10.42.0.1:8000/>
 - b. ALTERNATIVELY (local host): <http://127.0.0.1:8000/>
 - i. you can only do this if you are running the webpage from the RPi itself
 - c. troubleshooting in case neither option is working (or if on a different network):
 - i. in terminal: `ifconfig`
 - ii. find <inet> number associated with <wlan0>, that is your <IPv4>
 - iii. go to <http://<IPv4>:8000/>
3. Terminal shortcuts
 - a. to stop the application: `phstop`
 - b. to restart the application: `phrestart`
 - c. to view real-time logs in terminal: `phlogs`
 - d. to view the application status: `phstatus`
4. To make changes to the software
 - a. stop the application: `phstop`
 - b. make and save your edits (in your preferred IDE)
 - c. restart the application: `phrestart`
5. [OPTIONAL] To use mosquitto client terminal
 - a. Open new terminal
 - b. run the client in "listening" mode: `mosquitto_sub -h localhost -t "emg/#" -v`